# Music Mood DB: Functional Dependencies and Normalization

DBMS Final Project, Phase 1. This document lists the functional dependencies (FDs) for every table in the 19-table schema, checks each table against 2NF, 3NF and BCNF, and shows a worked decomposition from an unnormalized table.

Notation: `X -> Y` means that if two rows agree on X they must agree on Y. A **superkey** is any set of attributes that determines all attributes of the table. A **candidate key** is a minimal superkey.

## 1. How we found the FDs

1. Every primary key determines all other columns of its table (`PK -> all columns`).
2. Every UNIQUE column is a candidate key, so it also determines all other columns.
3. We then asked, for every non-key column, whether it is determined by anything other than a key. For example: does `country` determine `joined_at`? Does `plan_type` determine anything? In this domain the answer is no, so no extra FDs appear.

The assumptions behind the FDs are stated in section 4, so you can defend them in the viva.

## 2. FDs for each table

| Table | Candidate key(s) | Functional dependencies |
|---|---|---|
| `app_user` | `user_id`, `username`, `email` | `user_id -> username, email, first_name, last_name, birth_date, country, joined_at`; `username -> user_id`; `email -> user_id` |
| `free_user` | `user_id` | `user_id -> daily_skip_limit` |
| `premium_user` | `user_id` | `user_id -> plan_type, subscribed_on, valid_till` |
| `artist` | `artist_id` | `artist_id -> name, country, debut_year` |
| `artist_alias` | (`artist_id`, `alias`) | none besides the key (all attributes are in the key) |

| Table | Candidate key(s) | Functional dependencies |
|---|---|---|
| `album` | `album_id` | `album_id -> artist_id, title, release_date` |
| `track` | `track_id`, (`album_id`, `track_no`) | `track_id -> album_id, title, track_no, duration_sec, valence, energy, tempo_bpm`; `album_id, track_no -> track_id` |
| `genre` | `genre_id`, `name` | `genre_id -> name`; `name -> genre_id` |
| `mood` | `mood_id`, `name` | `mood_id -> name, parent_mood_id, description`; `name -> mood_id` |
| `track_artist` | (`track_id`, `artist_id`) | `track_id, artist_id -> role` |
| `track_genre` | (`track_id`, `genre_id`) | none besides the key |
| `track_mood` | (`track_id`, `mood_id`) | `track_id, mood_id -> confidence` |
| `user_mood_log` | `log_id` | `log_id -> user_id, mood_id, intensity, logged_at` |
| `listening_history` | `listen_id` | `listen_id -> user_id, track_id, played_at, duration_listened_sec` |
| `rating` | (`user_id`, `track_id`) | `user_id, track_id -> rating, rated_at` |
| `user_follow` | (`follower_id`, `followee_id`) | `follower_id, followee_id -> followed_at` |
| `playlist` | (`user_id`, `playlist_no`) | `user_id, playlist_no -> name, created_at, mood_id` |
| `playlist_track` | (`user_id`, `playlist_no`, `track_id`) | `user_id, playlist_no, track_id -> position, added_at` |
| `recommendation` | `rec_id` | `rec_id -> user_id, track_id, mood_id, score, generated_at` |

Note on `recommendation`: (`user_id`, `track_id`, `mood_id`, `generated_at`) is also a candidate key under the assumption that the system does not generate the same recommendation twice at the same instant. We keep `rec_id` as the primary key.

# 3. Normal form check

## 3.1 1NF

All columns are atomic. The composite attribute `name` was flattened into `first_name` and `last_name`, and the multivalued attribute `{alias}` was moved to `artist_alias`. Derived attributes (`age`, `play_count`, `completion_pct`, `track_count`, `total_duration`) are not stored, so no table holds computed or repeating data.

## 3.2 2NF (no partial dependencies)

2NF only matters where the key has more than one column. For those tables, every non-key attribute depends on the **whole** key:

| Table | Composite key | Non-key attributes | Result |
|---|---|---|---|
| `track_artist` | (`track_id`, `artist_id`) | `role` | `role` depends on the pair (one artist can be main on one track and featured on another). In 2NF |
| `track_mood` | (`track_id`, `mood_id`) | `confidence` | `confidence` is per tag. In 2NF |
| `rating` | (`user_id`, `track_id`) | `rating`, `rated_at` | Both belong to the pair. In 2NF |
| `user_follow` | (`follower_id`, `followee_id`) | `followed_at` | In 2NF |
| `playlist` | (`user_id`, `playlist_no`) | `name`, `created_at`, `mood_id` | None of them is determined by `user_id` alone or `playlist_no` alone. In 2NF |
| `playlist_track` | (`user_id`, `playlist_no`, `track_id`) | `position`, `added_at` | Position and added time are specific to the playlist-track pair. In 2NF |
| `artist_alias`, `track_genre` | whole row is the key | none | Trivially in 2NF |

All other tables have a single-column PK, so partial dependencies are impossible.

### 3.3 3NF (no transitive dependencies)

A transitive dependency would be `PK -> A -> B` where A is not a key. We looked for these:

- `track.album_id` does not determine `title`, `duration_sec` or any other track column. Album facts (`album.title`, `release_date`) live in `album`, which is why `album` is a separate table.
- `album.artist_id` does not determine anything in `album` besides itself. Artist facts live in `artist`.
- `listening_history` stores only `user_id` and `track_id`. User and track facts are not repeated there.
- `app_user.country` does not determine any other column.
- `premium_user.plan_type` does not determine `valid_till`: two students can have different end dates.
- `playlist.mood_id` does not determine `name`.

No transitive dependencies remain, so every table is in 3NF.

### 3.4 BCNF

BCNF requires that for every non-trivial FD `X -> Y`, X is a superkey. Looking at the table in section 2, every determinant on the left side is a candidate key or contains one:

- `app_user`: determinants are `user_id`, `username`, `email`, all candidate keys.
- `track`: determinants are `track_id` and (`album_id`, `track_no`), both candidate keys.
- `genre`, `mood`: determinants are the PK and the UNIQUE `name`, both candidate keys.
- All other tables have only the primary key as determinant.

**Result: all 19 tables are in BCNF.** Because every table is in BCNF, the decomposition is also lossless and dependency preserving.

## 4. Assumptions behind the FDs

These are the business rules that justify the FDs. Change an assumption and the FDs change.

| Assumption | FD it supports |
|---|---|
| A track belongs to exactly one album (singles count as a one-track album) | `track_id -> album_id` |
| An album belongs to exactly one (main) artist | `album_id -> artist_id` |
| Track number is unique inside an album | `album_id, track_no -> track_id` |
| A user rates a track at most once; a re-rating overwrites | `user_id, track_id -> rating, rated_at` |
| A user has at most one "follow" row per followee | `follower_id, followee_id -> followed_at` |
| Username and email are globally unique | `username -> user_id`, `email -> user_id` |
| Mood names and genre names are unique | `name -> mood_id`, `name -> genre_id` |
| A track appears at most once in a given playlist | `user_id, playlist_no, track_id -> position, added_at` |
| A user is either free or premium, never both | `user_id` is the PK of both subtype tables |

## 5. Worked example: normalizing a flat table

This is the kind of table the design replaces. It is useful to show how the FDs drive the decomposition.

### Unnormalized relation

`LISTEN_FLAT(listen_id, user_id, username, track_id, track_title, duration_sec, album_id, album_title, artist_id, artist_name, played_at, duration_listened_sec)`

### FDs

1. `listen_id -> user_id, track_id, played_at, duration_listened_sec`
2. `user_id -> username`
3. `track_id -> track_title, duration_sec, album_id`
4. `album_id -> album_title, artist_id`
5. `artist_id -> artist_name`

### Problems

- Candidate key is `listen_id`. FDs 2 to 5 have a non-key determinant (`user_id`, `track_id`, `album_id`, `artist_id`), so the relation is **not** in 3NF.
- Transitive chain: `listen_id -> track_id -> album_id -> artist_id -> artist_name`.
- Anomalies: renaming an artist means updating every listen row (update anomaly). A track with no listens cannot be stored (insertion anomaly). Deleting a user's only listen deletes the track's details (deletion anomaly).

**Attribute closure example**

Compute `{track_id}+`:

1. Start: `{track_id}`
2. FD 3 adds `track_title`, `duration_sec`, `album_id`
3. FD 4 (since `album_id` is now in the set) adds `album_title`, `artist_id`
4. FD 5 adds `artist_name`

So `{track_id}+ = {track_id, track_title, duration_sec, album_id, album_title, artist_id, artist_name}`. It does not include `listen_id` or `user_id`, so `track_id` is not a key of the flat relation.

**Decomposition (each step splits on an FD)**

| Step | Split on | New relations |
|---|---|---|
| 1 | FD 5 | `artist(artist_id, artist_name)` and the rest |
| 2 | FD 4 | `album(album_id, album_title, artist_id)` and the rest |
| 3 | FD 3 | `track(track_id, track_title, duration_sec, album_id)` and the rest |
| 4 | FD 2 | `app_user(user_id, username)` and the rest |
| 5 | remainder | `listening_history(listen_id, user_id, track_id, played_at, duration_listened_sec)` |

This gives exactly the `artist`, `album`, `track`, `app_user` and `listening_history` tables of the final schema. Each is in BCNF, and joining them on `user_id`, `track_id`, `album_id` and `artist_id` recreates `LISTEN_FLAT` with no extra or missing rows (lossless join).

## 6. Why derived attributes are not stored

If `age` were stored in `app_user`, then `birth_date -> age` would hold. `birth_date` is not a key, so this is a transitive dependency (`user_id -> birth_date -> age`) and violates 3NF. The same is true of `play_count` in `track` (determined by the rows of `listening_history`) and `track_count` in `playlist`. Computing them with functions and views keeps every table in 3NF and BCNF, and avoids stale values.

## 7. Canonical cover example: `track`

Given the FDs on `track`:

- F1: `track_id -> album_id, title, track_no, duration_sec, valence, energy, tempo_bpm`
- F2: `album_id, track_no -> track_id`

1. Split right sides into single attributes (F1 becomes 7 FDs).
2. No left side has an extra attribute: removing `album_id` or `track_no` from F2 breaks it, because neither alone identifies a track.
3. No FD is redundant: `track_id -> album_id` and `track_id -> track_no` are both needed to derive F2's left side, and F2 is needed to go back to `track_id`.

So the canonical cover has the same FDs, written with single attributes on the right side. This means `track` has two candidate keys and no further decomposition is needed.

## 8. Summary

| Normal form | Result |
|---|---|
| 1NF | Satisfied (composite flattened, multivalued split out, no derived columns stored) |
| 2NF | Satisfied (no partial dependencies in any composite-key table) |
| 3NF | Satisfied (no transitive dependencies) |
| BCNF | Satisfied (every determinant is a superkey) |
| Lossless join / dependency preserving | Yes |